

Module Federation: from browser mechanics to this lab shell

A visual path from “another build provides a module” to the exact admission, loading, activation, readiness, and mounting boundaries used by `demo-catalog`.

[general MF theory](#)[browser & Vite behavior](#)[verified in this repo](#)[runtime request sequence inference](#)

Keep the two questions separate. Federation answers how a host obtains a public module from another build. The shell answers whether metadata is admitted, where a contribution may appear, when it should load, what API it receives, and what failure state the UI should show.

This visual companion condenses [MODULE-FEDERATION-TUTORIAL.md](#). Online links teach the general mechanism; local links show the implementation actually used here.

01 Plain-language mental model

[general MF theory](#)[verified in this repo](#)

A **host** learns where a separately built **remote** lives. Its **remote entry** exposes a **container**; the container makes named public modules available. The federation **runtime** registers that location, resolves an exposure, and coordinates a **share scope** for libraries such as Vue. The export is ordinary JavaScript — the host decides whether to render it as a Vue component.

FEDERATION MENTAL MODEL — PUBLIC MODULE FROM ANOTHER BUILD



WHAT CHANGES FROM A NORMAL IMPORT?

```
import './Feature.vue'
```

host build knows and emits it

```
import('./Feature.vue')
```

lazy chunk, still host-owned

```
loadRemote('remote/plugin')
```

another build owns its module

Federation mental model. Federation is a boundary between build graphs, not a second browser. The remote code runs in the same page and JavaScript realm as the shell. See the [Module Federation concepts](#) for the general vocabulary; the local remote exposes `./plugin` in [demo-catalog's Vite config](#).

Host and remote are roles. A general federated app can be both. In this lab, `lab_shell` consumes and plugins expose.

Exposure is public.

`./src/plugin.js` is a remote implementation path; `./plugin` is the public key the host requests.

Not a sandbox. Catching a rejected load or render error is resilience, not isolation from same-page JavaScript.

02 What happens in the browser

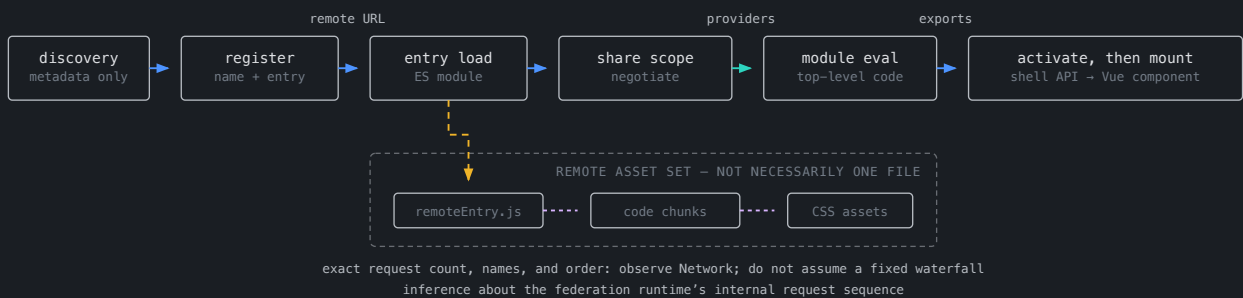
general MF theory

browser & Vite behavior

runtime request sequence inference

Loading an exposure is asynchronous. Discovery and registration decide *where* to look; the browser then loads the entry and any required assets, federation coordinates sharing, and the module evaluates before the shell receives its exports. The exact asset/chunk request order is a runtime-and-build implementation detail, not a stable protocol.

BROWSER LIFECYCLE — ONE FEATURE'S FIRST USE



Browser loading lifecycle. Entry loading is not feature completion. A Vite remote entry is explicitly registered as type: `'module'` in this shell's [federated adapter](#). Browser module evaluation is asynchronous; use DevTools Network to inspect the actual run. [MDN's import\(\) guide](#) and the [runtime API](#) describe the two layers.

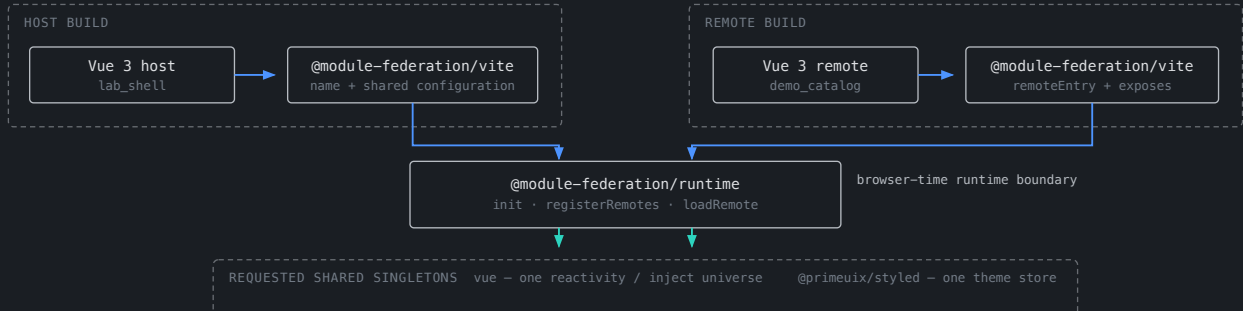
03 The local stack and its sharing contract

verified in this repo

general MF theory

The Vite plugin shapes each independent build. The runtime performs the late registration and module request in the page. Vue renders returned component definitions. This is deliberately not one responsibility called “federation.”

LOCAL STACK — BUILD-TIME INTEGRATION AND BROWSER-TIME LOADING



Local stack: build plugin vs runtime. Build plugin and runtime are different tools at different moments. Both [host](#) and [catalog](#) request singleton `vue` and `@primeuix/styled`. One Vue instance protects reactivity and dependency injection; sharing Vue alone would not share PrimeVue's styling store. [Shared configuration reference](#).

verified in this repo

`resolve.dedupe` in Vite is a related local build/test precaution; it

is not the browser share scope. Likewise, compatible federation libraries do not prove the application-level `shellApiVersion` is compatible.

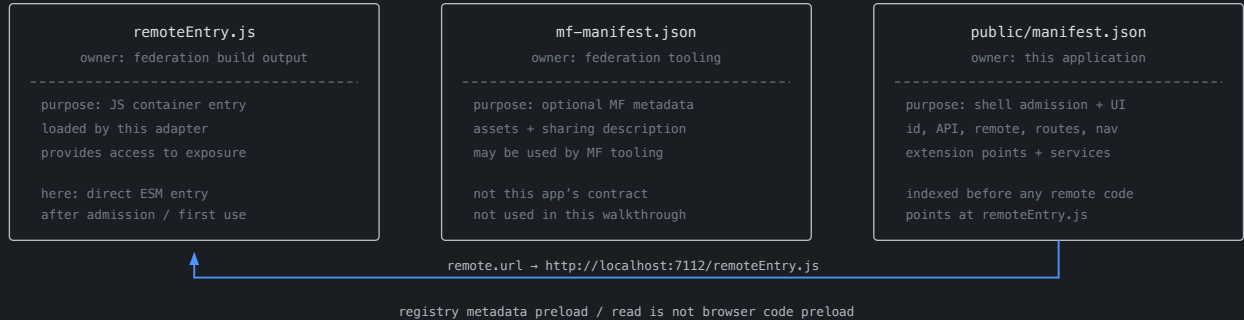
04 Three easily confused artifacts

general MF theory

verified in this repo

All three may be JSON-or-JavaScript-looking files near a remote. They do not have the same owner or job. The app-level manifest decides admission and placement; it happens before the direct remote entry is fetched.

THREE ARTIFACTS – SAME NEIGHBOURHOOD, DIFFERENT CONTRACTS



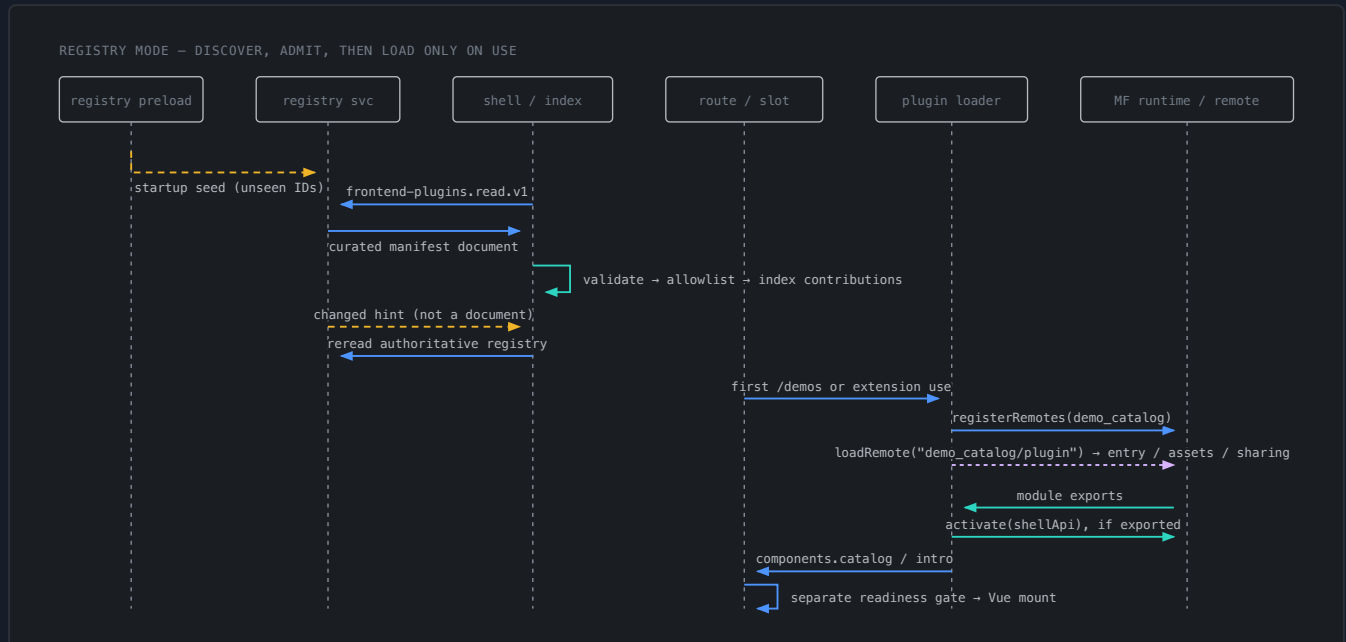
Three-artifact comparison. The catalog manifest is not a renamed federation manifest. [This repo's public manifest](#) carries application metadata and points to `remoteEntry.js`; the direct-entry adapter never needs an `mf-manifest.json`. Compare the [federation manifest fields](#) documentation with the local contract.

05 Exact demo-catalog walkthrough

verified in this repo

runtime request sequence inference

Default `registry` mode discovers curated manifests over the shell's NATS API. A changed notification is a reread hint, not the registry itself. Only after validation, contribution indexing, and a lazy route or extension request does the shell cross into federation. `build` mode is an alternate discovery source feeding the same admission and loading path.



demo-catalog sequence. Metadata can make a route visible before remote code exists in memory. The default source selection is in [pluginSource.js](#); reads and changed hints are in [registryTransport.js](#), [registrySession.js](#), and [changeSubscription.js](#). The purple request is deliberately tagged as an internal request-sequence inference, not a fixed network contract.

Admission first. [bootShell.js](#) validates manifests, adds allowed remote URLs, declares extension points, and indexes contributions without importing remote code.

Lazy first use. Lazy routes from [shellRoutes.js](#) ask the loader; the adapter normalizes `./plugin` to `plugin` and requests `demo_catalog/plugin`.

Activation, then mounting. [plugin.js](#) exports `catalog`, `intro`, and `activate`. [demoGate.js](#) checks readiness after loading but before mount.

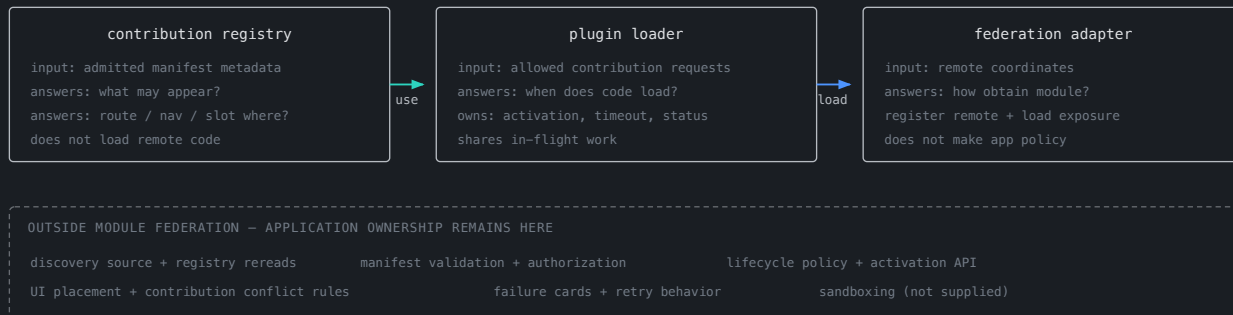
06 Responsibility boundaries

verified in this repo

general MF theory

The useful seam is not “everything about plugins.” Each layer has one question. That separation is why the shell can index features without downloading them, and why loader policy can be tested with an injected adapter.

SHELL RESPONSIBILITIES ENCLOSE THE FEDERATION MECHANISM



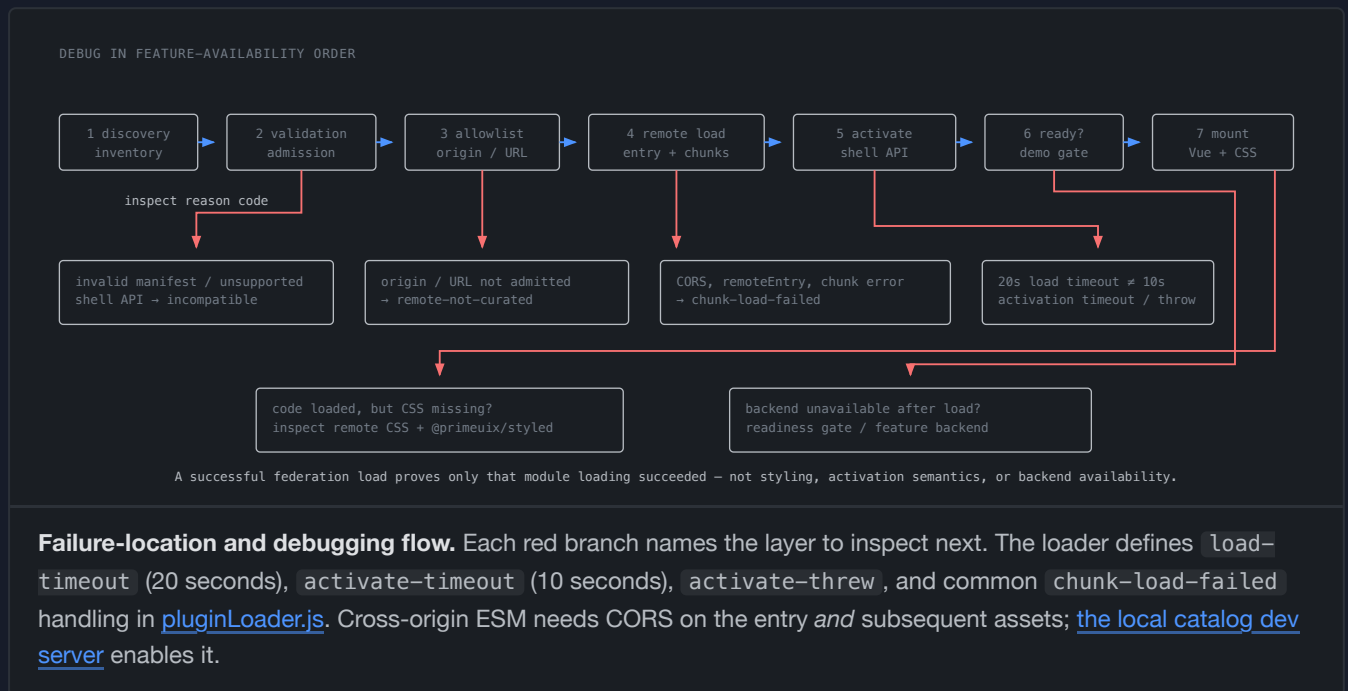
Responsibility-layer map. Federation is intentionally the narrowest layer. [Contribution registry](#) handles placement; [plugin loader](#) enforces allowlist, activation, status, and deadlines; [federated adapter](#) only obtains the exposure. Nothing about same-page federation is a security sandbox.

07 Failure and debugging guide

verified in this repo

browser & Vite behavior

Debug in the order a feature becomes available. Earlier failures mean later layers have no useful evidence yet: there is no value inspecting a chunk request for a manifest rejected before federation was called, or blaming federation for a backend which is unavailable only after code loaded.



1. **Find the plugin in the inventory.** Check source mode, discovery success, validation status, and the reason code before investigating browser requests.
2. **Check placement and policy.** Confirm the admitted route/component name and allowlisted remote URL; a non-admitted URL should not reach the adapter.
3. **Inspect the browser boundary.** Confirm container name, entry URL, public expose, CORS, ESM handling, entry/chunk responses, then distinguish a load deadline from activation.
4. **Only after code loads, check presentation and service state.** Separate CSS delivery from the shared styling store, and separate a successful module load from a failed pre-mount demo readiness check.

Reading path **Keep learning with the source beside the diagram**

General mechanism

1. [Module Federation concepts](#)
2. [Official Vite integration](#)
3. [Exposes](#) and [shared dependencies](#)
4. [Runtime troubleshooting](#)

Lab-shell implementation

1. [host Vite config](#) and [catalog Vite config](#)
2. [admission and indexing](#)
3. [runtime adapter](#) and [loader](#)
4. [pre-mount readiness gate](#)